

Week- 6

1# Write a C program that uses functions to perform the following operations on a circular

Linked List

- a. Creation
- b. Insertion
- c. Deletion
- d. Traversal.

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void insert_begin() {
    struct node *newnode;
    int value;
    newnode = (struct node *)malloc(sizeof(struct node));
    printf("Enter value: ");
    scanf("%d", &value);
    newnode->data = value;
    if (head == NULL) {
        head = newnode;
        newnode->next = head;
        newnode->prev = head;
    }
    else {
        newnode->next = head;
        newnode->prev = head->prev;
        head->prev->next = newnode;
        head->prev = newnode;
        head = newnode;
    }
}
```

```
===== CIRCULAR LINKED LIST =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter choice: 1

1.Beginning
2.Middle
3.End
1
Enter value: 4
Inserted at beginning.
```

```

    }
    printf("Inserted at beginning.\n");
}
void insert_end() {
    if (head == NULL) {
        insert_begin();
        return;
    }
    else {
        struct node *newnode, *temp;
        int value;
        newnode = (struct node *)malloc(sizeof(struct node));
        printf("Enter value: ");
        scanf("%d", &value);
        newnode->data = value;
        temp = head;
        while (temp->next != head) {
            temp = temp->next;
        }
        newnode->next = temp->next;
        temp->next->prev = newnode;
        temp->next = newnode;
        newnode->prev = temp;
    }
    printf("Inserted at end.\n");
}
void insert_middle() {
    if (head == NULL) {
        insert_begin();
        return;
    }
    struct node *newnode, *temp;
    int length = 1, i, value;
    newnode = (struct node *)malloc(sizeof(struct node));
    printf("Enter value: ");
    scanf("%d", &value);
    newnode->data = value;
    temp = head;
    while (temp->next != head) {
        temp = temp->next;
    }

```

```

===== CIRCULAR LINKED LIST =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter choice: 1

1.Beginning
2.Middle
3.End
2
Enter value: 43
Inserted in the middle

```

```

        length += 1;
    }
    temp = head;
    for (i = 1; i < length / 2; i++) {
        temp = temp->next;
    }
    newnode->next = temp->next;
    newnode->prev = temp;
    temp->next->prev = newnode;
    temp->next = newnode;
    printf("Inserted in the middle");
}

void delete_begin() {
    struct node *temp;
    if (head == NULL) {
        printf("List empty.\n");
        return;
    }
    temp = head;
    if (head->next == head) {
        head = NULL;
    }
    } else {
        head->prev->next = head->next;
        head->next->prev = head->prev;
        head = head->next;
    }
    free(temp);
    printf("First node deleted.\n");
}

void delete_end() {
    struct node *temp;
    if (head == NULL) {
        printf("List empty.\n");
        return;
    }
    temp = head;
    if (head->next == head) {
        head = NULL;
    }
}

```

```

Inserted in the middle
===== CIRCULAR LINKED LIST =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter choice: 3
List: Head<-> 4 <-> <-> 43 <-> Head

```

```

===== CIRCULAR LINKED LIST =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter choice: 2

1.Beginning
2.Middle
3.End
2
Node deleted at mid

```

```

    } else {
        temp = head->prev;
        temp->prev->next = temp->next;
        temp->next->prev = temp->prev;
    }
    free(temp);
    printf("Last node deleted.\n");
}

void delete_middle() {
    if (head == NULL) {
        printf("List empty.\n");
        return;
    }
    struct node *temp;
    int length = 1, i;
    temp = head;
    if (head->next == head) {
        head = NULL;
    }
    } else {
        while (temp->next != head) {
            temp = temp->next;
            length += 1;
        }
        temp = head;
        for (i = 0; i < (length + 1) / 2; i++) {
            temp = temp->next;
        }
        temp->prev->next = temp->next;
        temp->next->prev = temp->prev;
    }
    free(temp);
    printf("Node deleted at mid");
}

void traversal() {
    struct node *temp = head;
    if (head == NULL) {
        printf("List empty.\n");
        return;
    }
}

```

```

printf("List: ");
printf("Head<-> ");
while (temp->next != head) {
    printf("%d <-> ", temp->data);
    temp = temp->next;
}
printf("<-> %d <-> Head\n", temp->data);
}

int main() {
    int choice, subchoice;
    do {
        printf("\n===== CIRCULAR LINKED LIST =====\n");
        printf("1. Insertion\n");
        printf("2. Deletion\n");
        printf("3. Traversal\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("\n1.Beginning\n2.Middle\n3.End\n");
                scanf("%d", &subchoice);
                if (subchoice == 1)
                    insert_begin();
                else if (subchoice == 2)
                    insert_middle();
                else if (subchoice == 3)
                    insert_end();
                else
                    printf("Invalid option\n");
                break;
            case 2:
                printf("\n1.Beginning\n2.Middle\n3.End\n");
                scanf("%d", &subchoice);
                if (subchoice == 1)
                    delete_begin();
                else if (subchoice == 2)
                    delete_middle();
                else if (subchoice == 3)
                    delete_end();

```

```
        else
            printf("Invalid option\n");
            break;
    case 3:
        traversal();
        break;
    case 4:
        printf("Program ended.\n");
        break;
    default:
        printf("Invalid choice\n");
    }
} while (choice != 4);
return 0;
}
```

Week- 7

Q1- Write a C program that implements a stack and its operations using

a. Arrays

```
#include <stdio.h>
#define N 25
struct stacktype {
    int s[N];
    int top;
};
typedef struct stacktype stack;
int isempty(int top) {
    if (top == -1)
        return 1;
    return 0;
}
int isfull(int top) {
    if (top == N - 1)
        return 1;
    return 0;
}
void push(stack *sp, int item) {
    if (sp->top == N - 1) {
        printf("Stack full/overflow -Insertion not possible\n");
        return;
    }
    (sp->top)++;
    sp->s[sp->top] = item;
}
int pop(stack *sp) {
    int item;
    if (sp->top == -1) {
        printf("Stack empty/underflow -Deletion not possible\n");
        return 0;
    }
    item = sp->s[sp->top];
    (sp->top)--;
```

```
MENU (Stack Operations):
1.Push
2.Pop
3.Peek
4.Traversal
5.Exit
Choice : 1
Enter an int value to be pushed: 343
```

```

        return item;
    }
    int peek(stack st) {
        if (st.top == -1) {
            printf("Stack empty\n");
            return 0;
        }
        return st.s[st.top];
    }
    void traversal(stack st) {
        int i = 0;
        if (isempty(st.top)) {
            printf("\nStack empty\n");
            return;
        }
        printf("\nContents of Stack:\n");
        while (i <= st.top)
            printf("%d ", st.s[i++]);
        printf("<-top\n");
    }
    main() {
        int choice, item;
        stack st;
        st.top = -1;
        do {
            printf("\nMENU (Stack Operations):");
            printf("\n1.Push\n2.Pop\n3.Peek\n4.Traversal\n5.Exit\n");
            printf("Choice : ");
            scanf("%d", &choice);
            switch (choice) {
                case 1:
                    printf("Enter an int value to be pushed: ");
                    scanf("%d", &item);
                    push(&st, item);
                    break;
                case 2:
                    if (isempty(st.top))
                        printf("\nStack empty, deletion not possible\n");
                    else
                        {

```

```

MENU (Stack Operations):
1.Push
2.Pop
3.Peek
4.Traversal
5.Exit
Choice : 1
Enter an int value to be pushed: 23

```



```

        item = pop(&st);
        printf("Item popped= %d\n", item);
    }
    break;
case 3:
    if (isempty(st.top))
        printf("\nStack empty\n");
    else
    {
        item = peek(st);
        printf("Item at the top= %d\n", item);
    }
    break;
case 4:
    traversal(st);
    break;
}
} while (choice != 5);
}

```

b. Linked List (Pointers).

```

#include <stdlib.h>
struct nodetype {
    int INFO;
    struct nodetype *NEXT;
};
typedef struct nodetype node;
int isempty(node *top) {
    if (top == NULL)
        return 1;
    else
        return 0;
}
node *push(node *top, int item) {
    node *pnew = (node *)malloc(sizeof(node));
    pnew->INFO = item;
    pnew->NEXT = top;
    printf("Node inserted...\n");
    return pnew;
}
node *pop(node *top) {

```

MENU (Stack Operations):

1.Push
2.Pop
3.Peek
4.Traversal
5.Exit

Choice : 3

Item at the top= 23

```

node *pdel, *pnxt;
if (isempty(top))
{
    printf("\nStack empty -Deletion not possible\n");
    return top;
}
pdel = top;
pnxt = top->NEXT;
free(pdel);
printf("Node deleted...\n");
return pnxt;
}

int peek(node *top) {
    if (isempty(top))
    {
        printf("\nStack empty\n");
        return 0;
    }
    return top->INFO;
}

void traversal(node *top) {
    if (isempty(top))
        printf("\nStack empty\n");
    else
    {
        node *p = top;
        printf("\nContents of Stack:\nTop-> ");
        while (p != NULL)
        {
            printf("%d ", p->INFO);
            p = p->NEXT;
        }
        printf("\n");
    }
}

void main() {
    int choice, item;
    node *top = NULL;
    do {
        printf("\nMENU (Stack operations):");

```

MENU (Stack Operations):

```

1.Push
2.Pop
3.Peek
4.Traversal
5.Exit
Choice : 4

```

Contents of Stack:

```

343 23 <-top

```

```

printf("\n1.Push\n2.Pop\n3.Peek\n4.Traversal\n5.Exit\n");
printf("Choice : ");
scanf("%d", &choice);
switch (choice) {
case 1:
    printf("Enter int value to be pushed: ");
    scanf("%d", &item);
    top = push(top, item);
    break;
case 2:
    top = pop(top);
    break;
case 3:
    if (isempty(top))
        printf("\nStack empty\n");
    else
    {
        item = peek(top);
        printf("\nItem peeked= %d\n", item);
    }
    break;
case 4:
    traversal(top);
    break;
}
} while (choice != 5);
}

```

MENU (Stack Operations):

```

1.Push
2.Pop
3.Peek
4.Traversal
5.Exit
Choice : 2
Item popped= 23

```

MENU (Stack Operations):

```

1.Push
2.Pop
3.Peek
4.Traversal
5.Exit
Choice : 4

```

Contents of Stack:

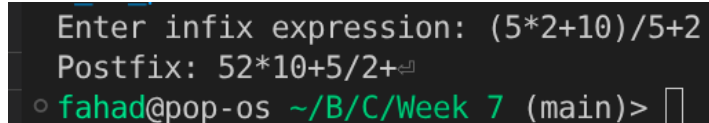
```

343 <-top

```

Q2- Write a C program to convert the Infix expression into a Postfix expression.

```
#include <stdio.h>
#include <ctype.h>
#define MAX 100
char stack[MAX];
int top = -1;
void push(char x) {
    stack[++top] = x;
}
char pop() {
    return stack[top--];
}
int precedence(char x) {
    if (x == '+' || x == '-') return 1;
    if (x == '*' || x == '/') return 2;
    if (x == '^') return 3;
    return 0;
}
int main() {
    char infix[MAX], postfix[MAX];
    int i = 0, j = 0;
    printf("Enter infix expression: ");
    scanf("%s", infix);
    push('(');
    strcat(infix, "");
    while (top != -1) {
        char ch = infix[i++];
        if (isdigit(ch)) {
            postfix[j++] = ch;
        }
        else if (ch == '(') {
            push(ch);
        }
        else if (ch == ')') {
            while (stack[top] != '(')
                postfix[j++] = pop();
            pop(); // remove '('
        }
    }
    postfix[j] = '\0';
    printf("Postfix: %s", postfix);
}
```

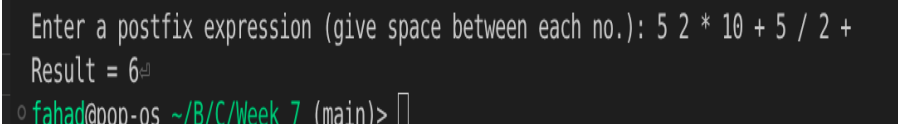


```
Enter infix expression: (5*2+10)/5+2
Postfix: 52*10+5/2+
fahad@pop-os ~/B/C/Week 7 (main)>
```

```
    }  
    else { // operator  
        while (precedence(stack[top]) >= precedence(ch))  
            postfix[j++] = pop();  
        push(ch);  
    }  
}  
postfix[j] = '\0';  
printf("Postfix: %s", postfix);  
return 0;  
}
```

Q3- Write a C program to evaluate a postfix expression.

```
#include <stdio.h>
#include <ctype.h>
#include <math.h>
#define MAX 100
int stack[MAX];
int top = -1;
void push(int x) {
    stack[++top] = x;
}
int pop() {
    return stack[top--];
}
int evaluatePostfix(char P[]) {
    int i = 0;
    int a, b, result;
    while (P[i] != '\0') {
        if (isdigit(P[i])) {
            push(P[i] - '0');
        }
        else {
            a = pop();
            b = pop();
            switch (P[i]) {
                case '+':
                    result = b + a;
                    break;
                case '-':
                    result = b - a;
                    break;
                case '*':
                    result = b * a;
                    break;
                case '/':
                    result = b / a;
                    break;
                case '^':
                    result = pow(b, a);
                    break;
            }
        }
        i++;
    }
    return result;
}
```



A terminal window with a dark background. The prompt is 'fahad@pop-os ~/B/C/Week 7 (main)>'. The user has entered the postfix expression '5 2 * 10 + 5 / 2 +'. The program has output 'Result = 6'.

```
Enter a postfix expression (give space between each no.): 5 2 * 10 + 5 / 2 +
Result = 6
fahad@pop-os ~/B/C/Week 7 (main)>
```

```
        push(result);
    }
    i++;
}
return pop();
}
int main() {
    char P[MAX];
    printf("Enter postfix expression: ");
    scanf("%s", P);
    int value = evaluatePostfix(P);
    printf("Value = %d", value);
    return 0;
}
```

Week- 8

Write a C program that implements Queue and its operations using

a. Arrays

```
#include <stdio.h>
#define MAX 1000
typedef struct {
    int arr[MAX];
    int front;
    int rear;
} Queue;
int is_empty(Queue q) {
    if ((q.front == -1 && q.rear == -1) || (q.front > q.rear))
        return 1;
    else
        return 0;
}
int is_full(Queue q) {
    if (q.rear == MAX - 1)
        return 1;
    else
        return 0;
}
void enqueue(Queue *q) {
    if (is_full(*q)) {
        printf("\nQueue full");
    }
    else if (is_empty(*q)) {
        q->front = 0;
        q->rear = 0;
        printf("\nEnter a value: ");
        scanf("%d", &(q->arr[q->rear]));
    }
    else {
        q->rear += 1;
        printf("\nEnter a value: ");
```

==== Queue ====

Choose an option:

1.Enqueue

2.Dequeue

3.Traversal

4.Exit

1

Enter a value: 24


```

        scanf("%d", &(q->arr[q->rear]));
    }
}
void dequeue(Queue *q) {
    if (is_empty(*q)) {
        printf("Cannot remove (Queue already empty)");
    }
    else {
        printf("Value dequeued is - %d", q->arr[q->front]);
        q->front += 1;
    }
}
void traversal(Queue q) {
    if (is_empty(q)) {
        printf("Queue is empty");
    }
    else {
        int i;
        for (i = q.front; i <= q.rear; i++) {
            printf("%d ", q.arr[i]);
        }
    }
}
int main() {
    Queue q1;
    q1.front = -1;
    q1.rear = -1;
    int choice;
    printf("\n==== Queue ==== \n");
    do {
        printf("\nChoose an option: ");
        printf("\n1.Enqueue\n2.Dequeue\n3.Traversal\n4.Exit\n");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                enqueue(&q1);
                break;
            case 2:
                dequeue(&q1);
                break;

```

Choose an option:

- 1.Enqueue
- 2.Dequeue
- 3.Traversal
- 4.Exit

1

Enter a value: 35

```

        case 3:
            traversal(q1);
            break;
        }
    } while (choice != 4);
}

```

b. Linked List (Pointers).

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

struct Node {
    int data;
    struct Node *next;
};
typedef struct Node Node;
struct Queue {
    Node *front;
    Node *rear;
};
typedef struct Queue Queue;
int is_queue_empty(Queue *queue) {
    if ((queue->front == NULL)) {
        return 1;
    }
    return 0;
}
Node *get_node() {
    Node *n = (Node *)malloc(sizeof(Node));
    n->next = NULL;
    return n;
}
Queue *get_queue(void) {
    Queue *q = (Queue *)malloc(sizeof(Queue));
    q->front = NULL;
    q->rear = NULL;
    return q;
}
void enqueue(Queue *q, int data) {
    Node *n = get_node();
    Node *temp = q->front;

```

Choose an option:

1.Enqueue

2.Dequeue

3.Traversal

4.Exit

3

24 35

```

n->data = data;
if (is_queue_empty(q)) {
    q->front = n;
}
else {
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = n;
}
q->rear = n;
}

int dequeue(Queue *q) {
    int del = q->front->data;
    Node *temp = q->front;
    if (q->front == q->rear) {
        q->front = NULL;
        q->rear = NULL;
    }
    else {
        q->front = temp->next; // q->front-next
    }
    free(temp);
    return del;
}

void traversal(Queue *q) {
    if (is_queue_empty(q)) {
        printf("Queue is empty");
    }
    else {
        printf("[ ");
        Node *temp = q->front;
        while (temp->next != NULL) {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("%d ", temp->data);
        printf("]");
    }
}

```

Choose an option:

- 1.Enqueue
- 2.Dequeue
- 3.Traversal
- 4.Exit

2

Value dequeued is - 24

```

int main() {
    int choice, data;
    Queue *q = get_queue();
    printf("\n==== Queue ==== \n");
    do {
        printf("\nChoose an option\n");
        printf("1.Enqueue\n2.Dequeue\n3.Traversal\n4.Exit\n> ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter a value :");
                scanf("%d", &data);
                enqueue(q, data);
                break;
            case 2:
                if (is_queue_empty(q))
                {
                    printf("Queue is already empty");
                }
                else {
                    int deleted = dequeue(q);
                    printf("Node dequeued = %d", deleted);
                }
                break;
            case 3:
                traversal(q);
        }
    } while (choice != 4);
}

```

```

Choose an option:
1.Enqueue
2.Dequeue
3.Traversal
4.Exit
3
35 53 24

```

Week- 9

Q1- Write a C program that implements a Circular Queue using arrays.

```
#include <stdio.h>
#define MAX 10
struct circular_queue {
    int arr[MAX];
    int rear;
    int front;
};
typedef struct circular_queue Queue;
int is_queue_empty(Queue *queue) {
    if (-1 == queue->rear)
    {
        return 1;
    }
    return 0;
}
int is_queue_full(Queue *queue) {
    {
        return ((queue->rear + 1) % MAX == queue->front);
    }
}
void enqueue(Queue *queue, int data) {
    {
        if (is_queue_empty(queue)) {
            queue->front = 0;
            queue->rear = 0;
        }
        else if (!(is_queue_full(queue))) {
            queue->rear = (queue->rear + 1) % MAX;
        }
        queue->arr[queue->rear] = data;
    }
}
int dequeue(Queue *queue) {
    {
        int data = queue->arr[queue->front];
```

Choose an option:

- 1.Enqueue
 - 2.Dequeue
 - 3.Traversal
 - 4.Exit
- 1

Enter a value: 23

```

    if (is_queue_empty(queue)) {
    }
    else if (queue->rear == queue->front) {
        queue->front = -1;
        queue->rear = -1;
    }
    else {
        queue->front = (queue->front + 1) % MAX;
    }
    return data;
}

void traversal(Queue *queue) {
    int index = queue->front;
    while (index != queue->rear) {
        printf("%d <- ", queue->arr[index]);
        index = (index + 1) % MAX;
    }
    printf("%d ", queue->arr[index]);
}

Queue get_queue() {
    Queue queue;
    queue.front = -1;
    queue.rear = -1;
    return queue;
}

int main() {
{
    Queue queue1 = get_queue();
    Queue *queue_pt = &queue1;
    int choice, data;
    printf("\n=== Queue ===");
    do {
        printf("\nChoose an option\n");
        printf("1.Enqueue\n2.Dequeue\n3.Traversal\n4.Exit\n> ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                if (is_queue_full(queue_pt))
                {
                    printf("\nQueue is full");

```

Choose an option:

1.Enqueue
2.Dequeue
3.Traversal
4.Exit

1

Enter a value: 35

Choose an option:

1.Enqueue
2.Dequeue
3.Traversal
4.Exit

3

23 63 57 58 35

```

    }
    else
    {
        printf("Enter a value :");
        scanf("%d", &data);
        enqueue(queue_pt, data);
    }
    break;
case 2:
    if (is_queue_empty(queue_pt))
    {
        printf("Queue is already empty");
    }
    else
    {
        int deleted = dequeue(queue_pt);
        printf("Node dequeued = %d", deleted);
    }
    break;
case 3:
    traversal(queue_pt);
    break;
default:
    printf("Invalid input");
}
} while (choice != 4);
}

```

```

Choose an option:
1.Enqueue
2.Dequeue
3.Traversal
4.Exit
2
Value dequeued is - 23

```

```

Choose an option:
1.Enqueue
2.Dequeue
3.Traversal
4.Exit
2
Value dequeued is - 63

```

Q2- Write a C program that uses both recursive and non-recursive functions to perform the following searching operations for a key value in a given list of integers:

a) Linear search

```
#include <stdio.h>

int linear_search_with_loop(int arr[], int size, int item) {
    int i = 0;
    while (i < size) {
        if (arr[i] == item)
            return i;
        i++;
    }
    return -1;
}

int linear_search_with_recursion(int arr[], int size, int item, int i) {
    if (i >= size) {
        return -1;
    }
    if (arr[i] == item) {
        return i;
    }
    return linear_search_with_recursion(arr, size, item, i + 1);
}

int main() {
    int n, i, choice, postion, item;
    printf("How many values you want to enter :");
    scanf("%d", &n);
    int arr[n];
    printf("\nEnter values :");
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    printf("Enter the digit you want to search :");
    scanf("%d", &item);
    printf("\nChoose and option\n");
```

```
How many values you want to enter :5

Enter values :4 63 15 57 13
Enter the digit you want to search :57

Choose and option
1.Search by loop
2.Search by recursion
> 1
Postion = 3
```



```

printf("1.Search by loop\n2.Search by recursion\n> ");
scanf("%d", &choice);
if (choice == 1) {
    postion = linear_search_with_loop(arr, n, item);
}
else if (choice == 2) {
    postion = linear_search_with_recursion(arr, n, item, 0);
}
else {
    printf("\nInvalid input");
}
if (postion == -1) {
    printf("\nItem not found");
}
else {
    printf("Postion = %d", postion);
}
}

```

How many values you want to enter :7

Enter values :54 25 6 24 642 643 6

Enter the digit you want to search :46

Choose and option

1.Search by loop

2.Search by recursion

> 2

Item not found

b) Binary search.

```
#include <stdio.h>
```

```
int binary_search_using_loop(int arr[], int size, int item) {
```

```
    int mid;
```

```
    int lb = 0;
```

```
    int ub = size - 1;
```

```
    while (lb <= ub) {
```

```
        mid = (lb + ub) / 2;
```

```
        if (arr[mid] == item) {
```

```
            return mid;
```

```
        }
```

```
        else if (arr[mid] > item) {
```

```
            ub = mid - 1;
```

```
        }
```

```
        else {
```

```
            lb = mid + 1;
```

```
        }
```

```
    }
```

```
}
```

```
int binary_search_using_recursion(int arr[], int lb, int ub, int item) {
```

```
{
```

```
    int mid = (lb + ub) / 2;
```

```
    if (arr[mid] == item) {
```

```
        return mid;
```

```
    }
```

```
    else if (arr[mid] > item) {
```

```
        ub = mid - 1;
```

```
        return binary_search_using_recursion(arr, lb, ub, item);
```

```
    }
```

```
    else {
```

```
        lb = mid + 1;
```

```
        return binary_search_using_recursion(arr, lb, ub, item);
```

```
    }
```

```
}
```

```
int main() {
```

```
{
```

```
    int n, i, choice, position, item;
```

```
    printf("How many values you want to enter :");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
How many values you want to enter :6
```

```
Enter values (in ascending order) :2 35 46 57 86 333
```

```
Enter the digit you want to search :57
```

```
Choose and option
```

```
1.Search by loop
```

```
2.Search by recursion
```

```
> 1
```

```
Position = 3
```

```

printf("\nEnter values :");
for (i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}
printf("Enter the digit you want to search :");
scanf("%d", &item);
printf("\nChoose and option\n");
printf("1.Search by loop\n2.Search by recursion\n> ");
scanf("%d", &choice);
if (choice == 1) {
    postion = binary_search_using_loop(arr, n, item);
}
else if (choice == 2) {
    postion = binary_search_using_recursion(arr, 0, n - 1, item);
}
else {
    printf("\nInvalid input");
}
if (postion == -1) {
    printf("\nItem not found");
}
else {
    printf("Postion = %d", postion);
}
}

```

```

How many values you want to enter :5

Enter values (in ascending order) :1 34 53 344 533
Enter the digit you want to search :32

Choose and option
1.Search by loop
2.Search by recursion
> 2

Item not found

```

Week- 10

Write a C program that implements the following sorting algorithms.

- a) Insertion sort for integers
- b) Bubble sort for characters
- c) Selection sort for integers
- d) Quick sort for characters

```
#include <stdio.h>
void insertion_sort(int arr[], int size) {
    int arr_ls, arr_fu, temp, pass;
    for (pass = 1; pass < size; pass++) {
        arr_fu = pass;
        arr_ls = pass - 1;
        temp = arr[arr_fu];
        while (arr_ls >= 0 && arr[arr_fu] < arr[arr_ls]) {
            arr[arr_ls + 1] = arr[arr_ls];
            arr_ls--;
        }
        arr[arr_ls + 1] = temp;
    }
}
void selection_sort(int arr[], int size) {
    {
        int temp, pass, i, local_min;
        for (pass = 0; pass < size; pass++) {
            local_min = pass;
            for (i = pass; i < size; i++) {
                if (arr[i] < arr[local_min]) {
                    local_min = i;
                }
            }
            if (local_min != pass) {
                temp = arr[pass];
                arr[pass] = arr[local_min];
            }
        }
    }
}
```

```
MENU
Choose an option
1. Insertion sort for integers
2. Bubble sort for characters
3. Selection sort for integers
4. Quick sort for characters
5. Exit
> 1
Enter the no. of elements you want to enter :6
Enter the elements : 53 743 7 35 64 46
Sorted array- 53 7 35 64 46 743
```

```

        arr[local_min] = temp;
    }
}

void int_traversal(int arr[], int size) {
{
    int i;
    printf("\nSorted array- ");
    for (i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
}

void char_traversal(char arr[], int size) {
{
    int i;
    printf("\nSorted array- ");
    for (i = 0; i < size; i++) {
        printf("%c ", arr[i]);
    }
}

void bubble_sort_for_char(char arr[], int size) {
{
    int pass, i, temp, changed;
    for (pass = 1; pass < size; pass++) {
        changed = 0;
        for (i = 0; i <= size - 1 - pass; i++) {
            if (arr[i] > arr[i + 1]) {
                temp = arr[i + 1];
                arr[i + 1] = arr[i];
                arr[i] = temp;
                changed = 1;
            }
        }
        if (!changed)
            break;
    }
}

void quicksort(char arr[], int lb, int ub) {
{
    int j;

```

Choose an option

1. Insertion sort for integers
2. Bubble sort for characters
3. Selection sort for integers
4. Quick sort for characters
5. Exit

> 2

Enter the no. of elements you want to enter :8

Enter the elements : q s c f t h n m

Sorted array- c f h m n q s t

```

        if (lb < ub) {
            j = partition(arr, lb, ub);
            quicksort(arr, lb, j - 1);
            quicksort(arr, j + 1, ub);
        }
    }

int partition(char arr[], int lb, int ub) {
    {
        int i, j, key, temp;
        i = lb + 1;
        j = ub;
        key = arr[lb];
        while (i <= j) {
            while (arr[i] < key && i <= j)
                i++;
            while (arr[j] > key)
                j--;
            if (i < j) {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
                i++;
                j--;
            }
        }
        else {
        }
        temp = arr[lb];
        arr[lb] = arr[j];
        arr[j] = temp;
        return j;
    }
}

int main() {
    {
        int choice, size, i;
        do {
            printf("\nMENU\n");
            printf("Choose an option\n");
            printf("1. Insertion sort for integers\n2. Bubble sort for characters\n3.
Selection sort for integers\n4. Quick sort for characters\n5. Exit\n> ");
            scanf("%d", &choice);

```

Choose an option

1. Insertion sort for integers
2. Bubble sort for characters
3. Selection sort for integers
4. Quick sort for characters
5. Exit

> 3

Enter the no. of elements you want to enter :6

Enter the elements : 34 6 364 67 23 53

Sorted array- 6 23 34 53 67 364

```

switch (choice) {
{
case 1:
{
    printf("Enter the no. of elements you want to enter :");
    scanf("%d", &size);
    int arr[size];
    printf("\nEnter the elements : ");
    for (i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }
    insertion_sort(arr, size);
    int_traversal(arr, size);
    break;
}
case 2:
{
    printf("Enter the no. of elements you want to enter :");
    scanf("%d", &size);
    char arr[size];
    printf("\nEnter the elements : ");
    for (i = 0; i < size; i++) {
        scanf(" %c", &arr[i]);
    }
    bubble_sort_for_char(arr, size);
    char_traversal(arr, size);
    break;
}
case 3:
{
    printf("Enter the no. of elements you want to enter :");
    scanf("%d", &size);
    int arr[size];
    printf("\nEnter the elements : ");
    for (i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }
    selection_sort(arr, size);
    int_traversal(arr, size);
    break;
}
}

```

```

Choose an option
1. Insertion sort for integers
2. Bubble sort for characters
3. Selection sort for integers
4. Quick sort for characters
5. Exit
> 4
Enter the no. of elements you want to enter :6

Enter the elements : g s e c h i

Sorted array-  c e g h i

```

```

    }
    case 4:
    {
        printf("Enter the no. of elements you want to enter :");
        scanf("%d", &size);
        char arr[size];
        printf("\nEnter the elements : ");
        for (i = 0; i < size; i++) {
            scanf(" %c", &arr[i]);
        }
        quicksort(arr, 0, size);
        char_traversal(arr, size);
        break;
    }
}
} while (choice != 5);
}

```


Week - 11

Write a C program that implements the following.

- a) Counting sort
- b) Merge sort
- c) Heap sort

```
#include <stdio.h>
void printArray(int arr[], int n) {
    printf("[ ");
    for (int i = 0; i < n; i++)
        printf("%d%s", arr[i], (i < n - 1) ? ", " : " ");
    printf("]\n");
}
void countingSort(int arr[], int n) {
    int max = arr[0], min = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) max = arr[i];
        if (arr[i] < min) min = arr[i];
    }
    int range = max - min + 1;
    int count[range];
    int output[n];
    for (int i = 0; i < range; i++) count[i] = 0;
    for (int i = 0; i < n; i++)
        count[arr[i] - min]++;
    for (int i = 1; i < range; i++)
        count[i] += count[i - 1];
    for (int i = n - 1; i >= 0; i--) {
        output[count[arr[i] - min] - 1] = arr[i];
        count[arr[i] - min]--;
    }
    for (int i = 0; i < n; i++)
        arr[i] = output[i];
}
void merge(int arr[], int left, int mid, int right) {
    int i = left;
```

```
SORTING ALGORITHMS - C

Enter the number of elements: 6
Enter 6 integer(s): 42 5 252 3 63 23

Original array : [ 42, 5, 252, 3, 63, 23 ]

Choose a Sorting Algorithm:
1. Counting Sort
2. Merge Sort
3. Heap Sort

Enter your choice (1-3): 1

Sorted array : [ 3, 5, 23, 42, 63, 252 ]
```

```

int j = mid + 1;
while (i <= mid && j <= right) {
    if (arr[i] <= arr[j]) {
        i++;
    } else {
        int value = arr[j];
        int k = j;
        while (k > i) {
            arr[k] = arr[k - 1];
            k--;
        }
        arr[i] = value;
        i++;
        mid++;
        j++;
    }
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;
    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {

```

```

SORTING ALGORITHMS - C

Enter the number of elements: 6
Enter 6 integer(s): 42 5 252 3 63 23

Original array : [ 42, 5, 252, 3, 63, 23 ]

Choose a Sorting Algorithm:
1. Counting Sort
2. Merge Sort
3. Heap Sort

Enter your choice (1-3): 2

Sorted array : [ 3, 5, 23, 42, 63, 252 ]

```

```

    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}

int main(void) {
    printf("
    printf("
    printf("
    int n;
    printf("Enter the number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid input. Please enter a positive integer.\n");
        return 1;
    }
    int arr[n];
    int workArr[n];
    printf("Enter %d integer(s): ", n);
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) {
            printf("Invalid input!\n");
            return 1;
        }
    }
    printf("\nOriginal array : ");
    printArray(arr, n);
    int choice;
    printf("
    printf("
    printf("
    printf("
    printf("Enter your choice (1-3): ");
    if (scanf("%d", &choice) != 1) {
        printf("Invalid choice!\n");
        return 1;
    }
}

```

```

SORTING ALGORITHMS - C

Enter the number of elements: 6
Enter 6 integer(s): 42 5 252 3 63 23

Original array : [ 42, 5, 252, 3, 63, 23 ]

Choose a Sorting Algorithm:
1. Counting Sort
2. Merge Sort
3. Heap Sort

Enter your choice (1-3): 3

Sorted array : [ 3, 5, 23, 42, 63, 252 ]

```

```
for (int i = 0; i < n; i++) workArr[i] = arr[i];
switch (choice) {
    case 1:
        countingSort(workArr, n);
        break;
    case 2:
        mergeSort(workArr, 0, n - 1);
        break;
    case 3:
        heapSort(workArr, n);
        break;
    default:
        printf("Invalid choice! Please select 1, 2, or 3.\n");
        return 1;
}
printf("\nSorted array : ");
printArray(workArr, n);
return 0;
}
```

Week- 12

Write a C program to perform the following operations:

- Insert an element into a binary search tree.
- Delete an element from a binary search tree.
- Search for a key element in a binary search tree.

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

struct Node *createNode(int value) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit;
    }
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

struct Node *insert(struct Node *root, int value) {
    if (root == NULL)
        return createNode(value);
    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    else
        printf(" [%d] %d already exists in the BST. Duplicates not allowed.\n", value);
    return root;
}

struct Node *findMin(struct Node *node) {
```

```
----- MENU -----
1. Insert an element
2. Delete an element
3. Search for a key
4. Display (In-order - sorted)
0. Exit
Enter your choice: 1
Enter value to insert: 35
[✓] 35 inserted.
```

```

while (node->left != NULL)
    node = node->left;
return node;
}
struct Node *deleteNode(struct Node *root, int value) {
    if (root == NULL) {
        printf(" [!] %d not found in the BST.\n", value);
        return NULL;
    }
    if (value < root->data) {
        root->left = deleteNode(root->left, value);
    }
    else if (value > root->data) {
        root->right = deleteNode(root->right, value);
    }
    else {
        if (root->left == NULL && root->right == NULL) {
            free(root);
            return NULL;
        }
        if (root->left == NULL) {
            struct Node *temp = root->right;
            free(root);
            return temp;
        }
        if (root->right == NULL) {
            struct Node *temp = root->left;
            free(root);
            return temp;
        }
        struct Node *successor = findMin(root->right);
        root->data = successor->data;
        root->right = deleteNode(root->right, successor->data);
    }
    return root;
}
struct Node *search(struct Node *root, int key) {
    if (root == NULL || root->data == key)
        return root;
    if (key < root->data)

```

```

----- MENU -----
1. Insert an element
2. Delete an element
3. Search for a key
4. Display (In-order - sorted)
0. Exit
Enter your choice: 4
In-order    : 35 42 63

```

```

        return search(root->left, key);
    else
        return search(root->right, key);
}
void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
void freeTree(struct Node *root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

```

```

int main(void) {
    struct Node *root = NULL;
    int choice, value;
    printf("=====\n");
    printf("  Binary Search Tree (BST) Operations  \n");
    printf("=====\n");
    do {
        printf("\n----- MENU ----- \n");
        printf("1. Insert an element\n");
        printf("2. Delete an element\n");
        printf("3. Search for a key\n");
        printf("4. Display (In-order - sorted)\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                root = insert(root, value);
                printf(" [✓] %d inserted.\n", value);
                break;

```

```

----- MENU -----
1. Insert an element
2. Delete an element
3. Search for a key
4. Display (In-order - sorted)
0. Exit
Enter your choice: 3
Enter key to search: 42
[✓] Key 42 FOUND in the BST.

```

```

case 2:
    if (root == NULL) {
        printf(" [!] BST is empty.\n");
        break;
    }
    printf("Enter value to delete: ");
    scanf("%d", &value);
    root = deleteNode(root, value);
    printf(" [✓] Delete operation completed for %d.\n", value);
    break;
case 3:
    if (root == NULL) {
        printf(" [!] BST is empty.\n");
        break;
    }
    printf("Enter key to search: ");
    scanf("%d", &value);
    {
        struct Node *result = search(root, value);
        if (result != NULL)
            printf(" [✓] Key %d FOUND in the BST.\n", value);
        else
            printf(" [X] Key %d NOT FOUND in the BST.\n", value);
    }
    break;
case 4:
    printf("In-order : ");
    inorder(root);
    printf("\n");
    break;
case 0:
    printf("Freeing memory and exiting...\n");
    freeTree(root);
    root = NULL;
    break;
default:
    printf(" [!] Invalid choice. Please try again.\n"); }
} while (choice != 0);
printf("Goodbye!\n");
return 0;}

```

```

----- MENU -----
1. Insert an element
2. Delete an element
3. Search for a key
4. Display (In-order - sorted)
0. Exit
Enter your choice: 2
Enter value to delete: 35
[✓] Delete operation completed for 35.

```